School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00170371
Student Name:	Alexandru Belecci

Module Information

Module Title:	Fundamentals of AI & Data Science
Module Code:	COMPH2706

Assessment Information

Assessment Title:	Project

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/

☑ I have completed the TU Dublin AI Driving License theory test.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	Belecci
Date:	20/04/2026

Content

- Executive Summary
- Introduction & Business Understanding
- Data Understanding & Exploration
- Methodology
- Results
- Conclusion
- References

Executive Summary

The primary objective of the project is to develop and evaluate machine learning models that can classify the Network Traffic into Benign (legitimate) and Brute Force (malicious). Analysis and Evaluation are done over a data set that holds 6000 network sessions which are classified as Benign or Brute Force which also holds an imbalance between them making the models perform worse due to the problem.

3 models will be used:

- Decision Tree
- Random Forest
- Tuned Random Forest
- Naïve Bayes

The most critical finding of the project is that models perform poorly in detection of Brute Force activity, a direct consequence of the Class Imbalance problem. Making them insufficient for analyzing on their own.

Introduction & Business Understanding

For ensuring a structured and systemic approach the project is following the CRISP-DM framework for Data Analysis and Model Development.

This report focuses on analysis of a Business' Cybersecurity Problem related to network intrusion detection system. This problem is critical as cyberattacks mostly result in massive financial and loyalty losses, operational disruptions, data breaches, etc. Meanwhile the system is overwhelmed managing the false positive alarms, the real attacks pass unnoticed and build their way through the system.
The development of a proper functional Network Intrusion Detection System which will be able to identify possible threats and predict network behaviour minimizing false positives remains the main objective and the best case scenario represents the high recall for attacks.

The dataset for analysis consists of network traffic data, 6000 records and 19 features which include both numerical and categorical. attack_category for classification and avg_packet_size for regression represent the main target features. Dataset also contains an imbalance for attack_category as 'Benign' type occurs twice more than 'Brute Force' which reflects real world scenarios.

Data Understanding & Exploration

Dataset Overview

The used project dataset holds 6000 records and 19 features, including both numerical and categorical, which represents aspects of different network traffic activities met in a day-to-day routine in a network eg. Packet size, protocol type, login attempts, port number, bytes sent etc.

For classification, attack_category is analysed as the target feature, which is the identifier for different types of cyberattacks based on symptoms presented, in this case Benign or Brute Force.
For regression avg_packet_size as the target feature, which represents the predicted network traffic behaviour. This is important in cybersecurity as packet size can indicate if any suspicious activity can be identified and as well to understand traffic patterns.

Statistical Summary

Data Completeness

For data completness visualization statistical analysis was performed and understood that, while majority of features are fully populated, containing 6000 records/feature, the following features are missing the exact number of records 420:

- protocol
- bytes_received
- signal_strength

Only containing 5580 non-null entries.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6000 entries, 0 to 5999
Data columns (total 19 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   packet_session_id  6000 non-null   object
 1   protocol           5580 non-null   object
 2   device_type        6000 non-null   object
 3   vpn_status         6000 non-null   object
 4   region             6000 non-null   object
 5   login_attempts     6000 non-null   int64
 6   failed_logins      6000 non-null   int64
 7   bytes_sent         6000 non-null   float64
 8   bytes_received     5580 non-null   float64
 9   packet_rate        6000 non-null   float64
 10  connection_count   6000 non-null   int64
 11  tcp_flags          6000 non-null   object
 12  port_number        6000 non-null   int64
 13  session_retries    6000 non-null   int64
 14  signal_strength    5580 non-null   float64
 15  privilege_level    6000 non-null   object
 16  anomaly_score      6000 non-null   float64
 17  avg_packet_size    6000 non-null   float64
 18  attack_category    6000 non-null   object
dtypes: float64(6), int64(5), object(8)
memory usage: 890.8+ KB
```

After a deeper analysis was performed, a key finding was gathered. That these 3 features' missing values don't rellate to each other and indicate that missing data is completely random, and only 2 rows contain missing values across all 3 features.

```
Rows missing in ALL three columns:  2
Rows missing in ANY of three columns: 1160
```

Data Standardisation

Feature 'protocol' is holding 5 unique values:

```
protocol                : 5 unique | sample: ['TCP' 'UDP' 'tcp' 'ICMP' 'udp']
```

But there are only 3 actual real values:
TCP, UDP and ICMP

This happened due to non-standardized string casing, and if uncorrected this can lead to broken analysis, providing inaccurate results.

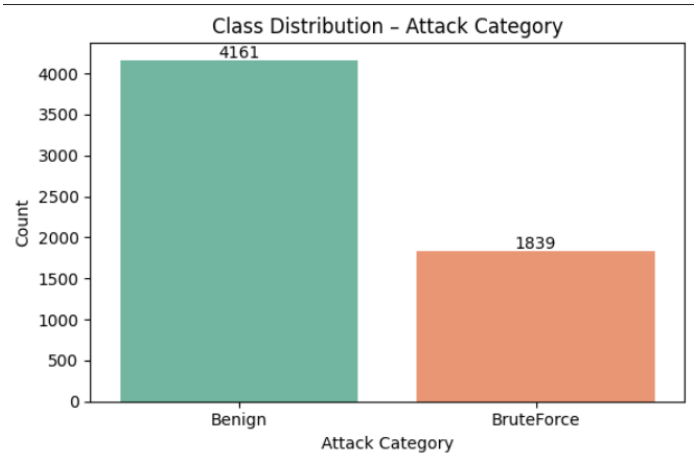
Feature packet_session_id is the unique identifier of each entry, containing 6000 unique values. It does not provide any useful information.

```
packet_session_id        : 6000 unique
```

Class Distribution

The target feature consists of two classes:

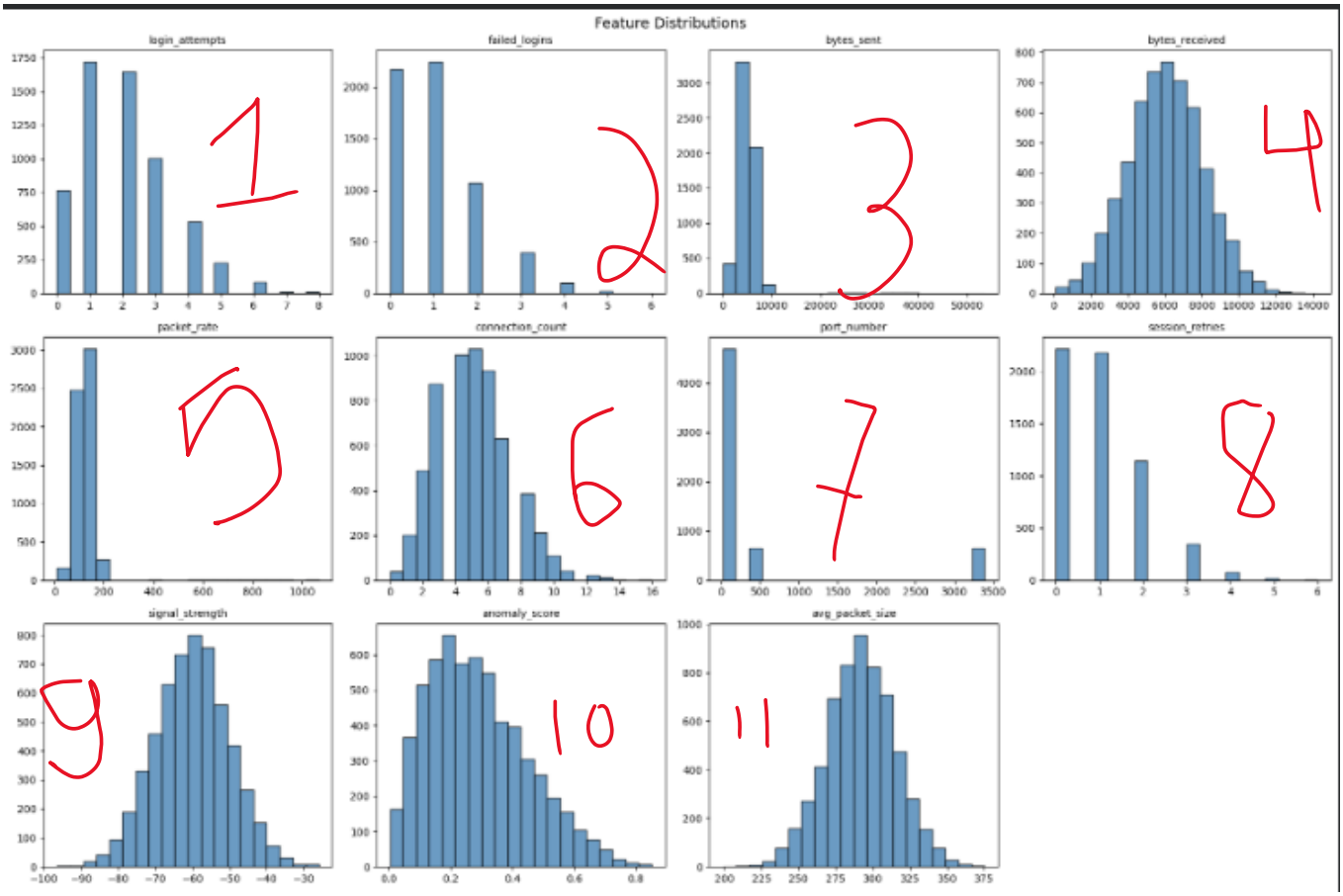
- Benign ~70% – consists of 4161 samples of the dataset.
- Brute Force ~30% – consists of 1839 samples of the dataset.



Noticed that Benign is the dominant in the Attack Category Class holding twice as many samples as Brute Force. Dataset represents a moderate class imbalance between 2 traffics.

In this case if unmodified, the model can predict the majority class. This can lead to missed Brute Force attacks detections, predicting false negatives, which is critical and could carry major consequences.

Feature Distributions



login_attempts (1) and failed_logins (2), both features' majority records are low counts 0 – 2, which are characteristics of Benign Traffic. Making them both Right-Skewed.

bytes_sent (3) represents a strong Right Skewed Distribution, where the majority network actors send between ~3000 and ~6000 bytes, a small bit are reaching and crossing the amount of 10000 bytes sent and around ~400 fall into 0 – ~3000 bytes range and no records occupy the other side of the histogram.

bytes_received (4), signal_strength (9) and avg_packet_size (11) are representants of normal/uniform distributions.
-bytes_received (4) distribution is mostly centered between ~5000 and ~7000, remarking the volume of received data across users on the network. Because there is no significant skew in any direction, it represents a well-behaved and stable feature.
-signal_strength (9) a normal distribution centered at ~ -73 and ~ -48 with a peak frequency of nearly 800 counts.
It touches both ends with weaker signals representing a realistic behaviour of wireless signals across the network.

Outliers are missing from the dataset of this feature.

-avg_packet_size (11) uniform distribution, centered at ~275 and ~310 and a peak frequency of nearly 1000 counts, which represents the most frequent average packet sizes flying across network. No notable outliers present in the histogram, giving the information that both classes produce the same packet sizes.

packet_rate (5) a clearly Right Skewed distribution, as the majority is concentrated between 0 and 200, and frequency of maximum 3000. Beyond ~220 frequences are stable at 0 which proves that smaller packet rates are dominating the feature representing normal network traffic behaviours.

connection_count (6) and anomaly_score (10) both roughly Right-Skewed.
-connection_count (6) centered at ~4-6 peaking at 1000, and frequences decline as connection_count increases and passes 10 connections. The 12-14 range may represent the potential outliers which may indicate port scanning attempts or Brute Force attacks.
-anomaly_score (10) centered at ~1.3 and ~3.6, with values spread between 0.0 and 0.8. Frequences hold at around 550-560 during centered time and the peak crosses way over 600. Also, a noticeable decline is seen beyond 0.4 in frequency in the feature thinning beyond 0.8, and the broad range can serve as valuable input for classification, because it may help the model in critical decisions.

port_number (7) remarks that port numbers are distributed around specific port numbers, at extremes of the histograms but the majority sessions stay at lower port numbers between 0 and 500 with highest frequency crossing 4000. Likely port numbers to be used by each session are:

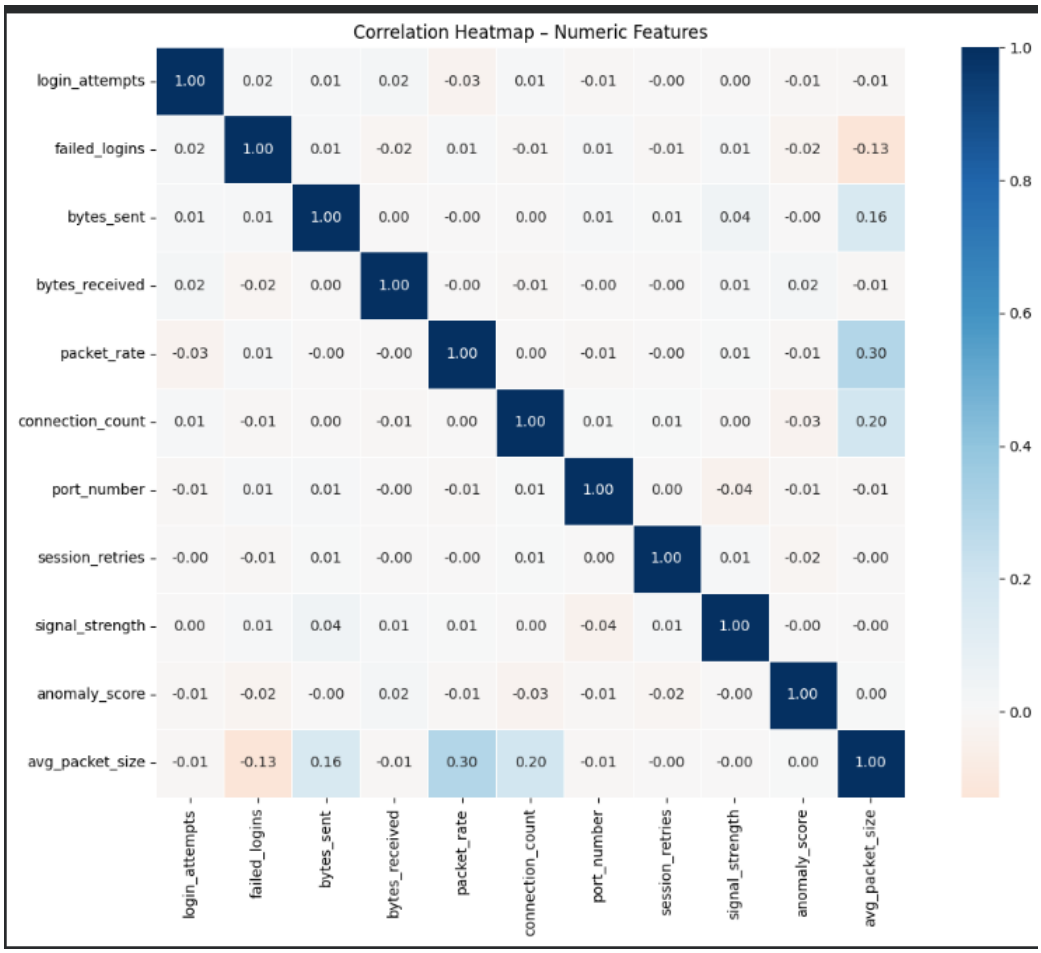
- 22 SSH
- 443 HTTPS
- 80 HTTP

Also, usage of higher ports between ~3200 and ~3400 is visible with a lower frequency, suggesting a small need for them. May be:

- Probation Ports
- Custom Services
- Suspicious Activity

session_retries (8) a Right Skewed distribution, indicating the vast majority of retries at 0 and 1 counts peaking at approximately 2300 sessions. A noticeable decrease happens at 2 retries, it drops by approximately twice the frequency and even more while it reaches 3, 4 and 5 retries/session. This indicates that majority of sessions connect successfully to first attempts, a higher number of sessions retries should be investigated as automated tools for Brute Force activity involve the same symptoms.

Correlation Heatmap

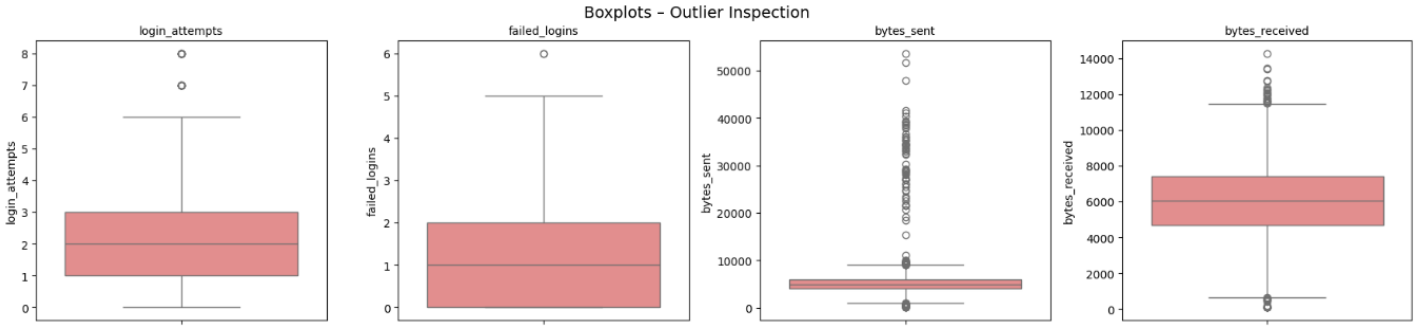


Heatmap displays the correlation between all features from the data set. Key Observations:

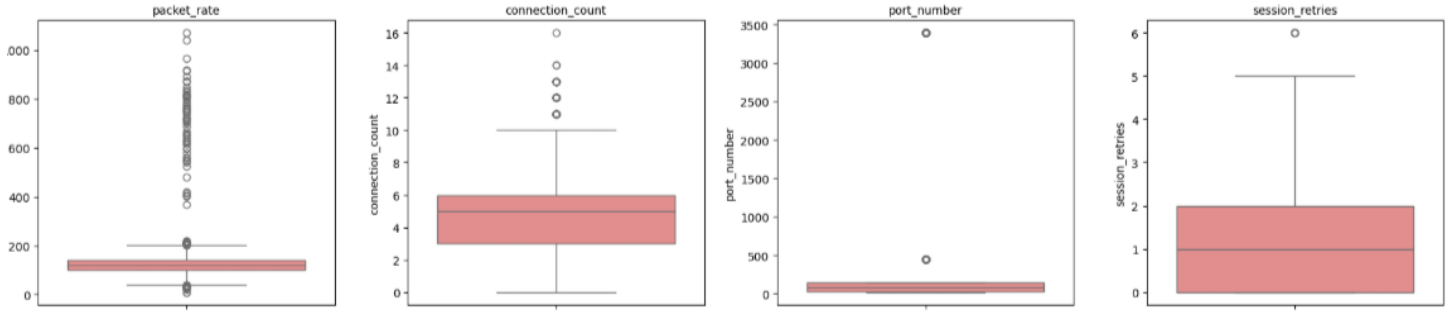
- Majority correlations are nearly 0, meaning that features are independent.
- A correlation of 0.30 is between packet_rate and avg_packet_size, as the packet rate may increase or decrease the average packet size. The highest correlation.
- A correlation of 0.20 is between connection_count and avg_packet_size, because of more connections happening in the session the data payload is increasing.
- A correlation of 0.16 is between bytes_sent and avg_packet_size, because the larger data is transmitted it will increase the average size of the packets in sessions.
- A negative correlation of -0.13 is between failed_logins and avg_packet_size, indicating that failed logins attempts are associated with smaller data payloads and it decreases the average overall size.

A good sign is that features are independent from each other operating with unique information for the model.

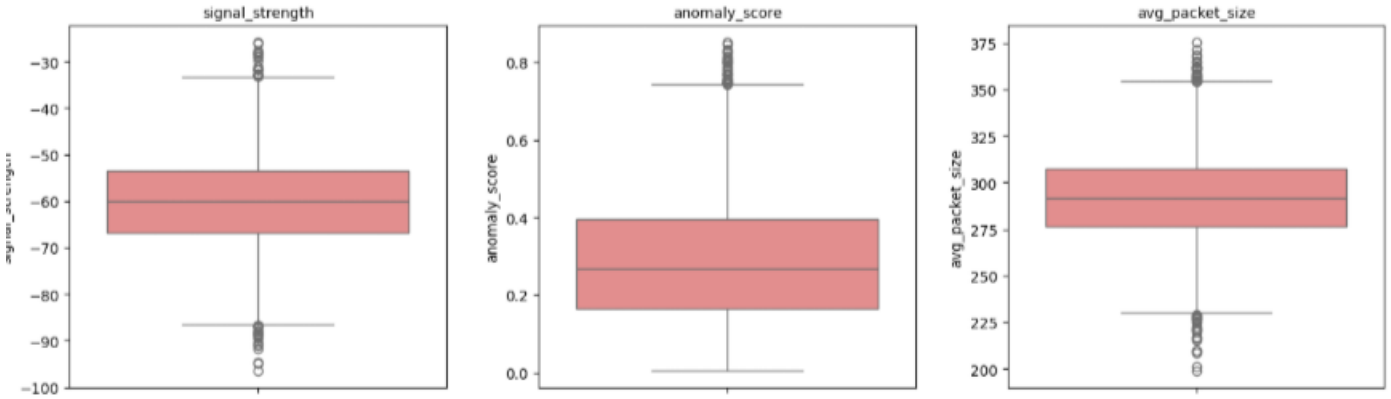
Outlier Inspection



- login_attempts
  The interquartile ranges from 1 to 3 with the median at 2 login attempts, also outliers are visible at 7 and 8 attempts. These outliers may represent Brute Force activity, because they represent unusual high login activity. Further investigation should be carried out.
- failed_logins
  The interquartile ranges from 0 to 2 with the median residing at 1 failed login, also outliers are visible at 6 failed login attempts. Similar to login_attempts, the noticeable behaviour of Brute Force activity representing an extreme case of repeated failed login attempts.
- bytes_sent
  The interquartile ranges from ~4000 to ~6000 bytes, and the median resides at ~5000 bytes. Majority of outliers reside between 40000 and 20000, but also a few of them cross the 50000 bytes. These outliers may represent sessions involved in large data transfers or could belong to Brute Force activity (high byte count through repeated attempts).
- bytes_received
  The interquartile ranges from ~5000 to ~6000 bytes, and the median resides at 6000 bytes. Compared to bytes_sent, outliers are fewer and less extreme and main outliers reside between ~ 12000 and ~13000 and only one is crossing the 14000 mark. These represent the sessions which received unusual payloads of data.



- packet_rate
  The interquartile ranges from ~100 to ~160 with the median residing at approximately 130, indicating that majority of sessions operate at lower packet rates. A vast number of outliers are present starting from 200 and even crossing the 1000 mark of rates. As a standalone feature for analyzing, it is not enough to make decisions as such packet rates could belong to Benign traffic or even Brute Force activity.
- connection_count
  The interquartile ranges from 3 to 6, with the median residing at 5 connections. A small number of outliers is visible spread evenly from 11 to 16 counts. These outliers should be taken into further investigation as may represent Brute Force activity.
- port_number
  The interquartile ranges from ~0 to ~200 with the median residing at ~100 representing the ports mostly in use which are the smaller number ports. However, a few outliers could be seen, near 500 and near 3500 port numbers, which as stated before in the histogram can indicate many findings, but further investigation should be undertaken.
- session_retries
  The interquartile ranges from 0 to 2 and the median resides at 1, representing the session retry attempts. An extreme session retry attempt is noticed at count 6 which can indicate a potential Brute Force activity and should be noted.



- signal_strength
  The interquartile ranges from ~-65 to ~-55, and the median resides at -60 representing a consistent central range. Unusual weak and strong session signals can be spotted by looking at the outliers populating from ~-35 and up and from ~-85 and down, not a standalone feature for future analysis.
- anomaly_score
  The interquartile ranges from nearly 0.2 to 0.4 with a median residing at ~0.27. The presence of only higher quartile points that a low anomaly score is expected, and only higher quartile outliers feel unusual and worth investigating, noticed a vast number of anomalies around 0.8 score.
- avg_packet_size
  The interquartile ranges from 275 to ~310 packet size, with a median residing at approximately 290. It represents the typical range of packet sizes met on a network, as well with many outliers present at both edges. bytes_sent, packet_rate and connection_count together with avg_packet_size collectively describe same network behaviour.

Methodology

Feature Engineering

```
df_clean['failed_login_ratio'] = df_clean['failed_logins'] / (df_clean['login_attempts'] + 1)
```

The new feature displays the proportion of failed logins over the total login attempts; it provides more information rather than number of failed logins by itself.

- 3 failed logins out of 10 is less suspicious than 3 failed logins out of 3.

+ 1 is added for avoiding division by 0

```
df_clean['anomaly_per_packet'] = df_clean['anomaly_score'] / (df_clean['packet_rate'] + 1)
```

The new feature represents the rate of anomaly per packet rate, giving feedback on how suspicious the network activity might be.

- A session might have an anomaly score of 0.8 and the packet rate 10000, in the end the anomaly per packet is 0.8/ (10000 + 1) ~ 0.00008 meaning that not each packet is suspicious.
- A session might have an anomaly of 0.6 and the packet rate is 3, in the end the anomaly per packet is 0.6 / (3 + 1) = 0.15, anomaly per packet is much higher = much suspicious.

	failed_login_ratio	anomaly_per_packet
count	6000.0000	6000.0000
mean	0.4312	0.0026
std	0.5405	0.0019
min	0.0000	0.0000
25%	0.0000	0.0013
50%	0.3333	0.0022
75%	0.6000	0.0034
max	6.0000	0.0653

failed_login_ratio
mean: ~43% login attempts fail in sessions.
std: ~54% values are spread out (min 0 and max 6)

anomaly_per_packet
mean: ~0.26% very low, sessions not suspicious
std: ~0.19% values are consistent, same behaviour (min 0 and max 0.0653)

Data Processing

Drop useless features

```
df_clean = df.copy()
df_clean.drop(columns=['packet_session_id'], inplace=True)
```

packet_session_id is the unique identifier, therefore it is not helping in prediction and should be dropped.

Standardize casing

```
# Standardize casing to Uppercase
df_clean['protocol'] = df['protocol'].str.upper()
```

As analyzed earlier, protocol was not standardized for future machine learning. In order for increasing accuracy and eliminating categorical fragmentation string standardization is involved

Handle missing values

```
# Handle missing values
df_clean.loc[:, 'protocol'] = df_clean['protocol'].fillna(df_clean['protocol'].mode()[0])
df_clean.loc[:, 'bytes_received'] = df_clean['bytes_received'].fillna(df_clean['bytes_received'].median())
df_clean.loc[:, 'signal_strength'] = df_clean['signal_strength'].fillna(df_clean['signal_strength'].median())
```

Past analysis showed that 420 entries for: protocol, bytes_sent and signal_strength were missing, this should be fixed by filling the missing values with the mode and median of each feature for minimizing the introduction of variance.

Label encoder

```
# Label encode all categorical columns except target
cat_cols = ['protocol','device_type','vpn_status','region','tcp_flags','privilege_level']
le_dict = {}
for col in cat_cols:
    le = LabelEncoder()
    df_clean[col] = le.fit_transform(df_clean[col].astype(str))
    le_dict[col] = le
```

Label encoder is applied at the following features so the model can work with them:

- Protocol
- Device_type
- Vpn_status
- Region
- Tcp_flag
- Privilege_level

```
# Encode target
le_target = LabelEncoder()
df_clean['attack_category_enc'] = le_target.fit_transform(df_clean['attack_category'])
print("Target encoding:", dict(zip(le_target.classes_, le_target.transform(le_target.classes_))))

Target encoding: {'Benign': np.int64(0), 'BruteForce': np.int64(1)}
```

After encoding the target feature, Benign traffic is taking the value: 0 and Brute Force traffic: 1 storing in another feature attack_category_enc

Feature scaling

```
# StandardScaler — zero mean, unit variance
feature_cols = [c for c in df_clean.columns if c not in ['attack_category','attack_category_enc']]
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(df_clean[feature_cols]), columns=feature_cols)
print(f"Scaled {len(feature_cols)} features. NaNs remaining: {X_scaled.isnull().sum().sum()}")
```

For ensuring that each feature will contribute for future decisions, they should be brought on the same scale so that larger numbers do not dominate.

Train/Test split

After data preparation, target column is extracted and is shuffled at a random state 42. Then data is split into X_train/test and y_train/test with 80% of data transferred for training and remaining 20% kept for testing.

```
y_clf = df_clean['attack_category_enc'].values
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    X_scaled, y_clf, test_size=0.2, random_state=42, stratify=y_clf)
```

Model development

For Machine Learning evaluation in detection of network traffic types three models were performed and compared.

Decision Tree (Baseline)
Trained using default parameters for future reference against improved models.
Chosen because it is the very basic method - baseline, that is establishing the future minimum acceptable performance that will be analyzed against other better models.

Random Forest (Improved)
Also trained with default parameters, for comparison of different approaches.
It is a group of multiple Decision Trees (Baselines), that is supposed to perform a better prediction than the Baseline model, because a group of Baselines perform.

Random Forest Grid Search (Tuned)
The most sophisticated model used for this report, where grid search was applied to the Random Forest across 27 parameters using 5-fold cross validation.
Expected to perform a better prediction than the normal Random Forest, because of the optimization.

Results

Model 1 Decision Tree

First model achieved an overall accuracy of 56%, Benign f1-score at 68% and Brute Force at 33% only. Making the model unreliable for attack detection, missing most Brute Force attacks.

Brute Force
Precision – 31% predicted sessions were Brute Force.
Recall – 35% attacks caught.

A high number of false negatives – 241 unpredicted attacks and only 127 predicted out of 368.

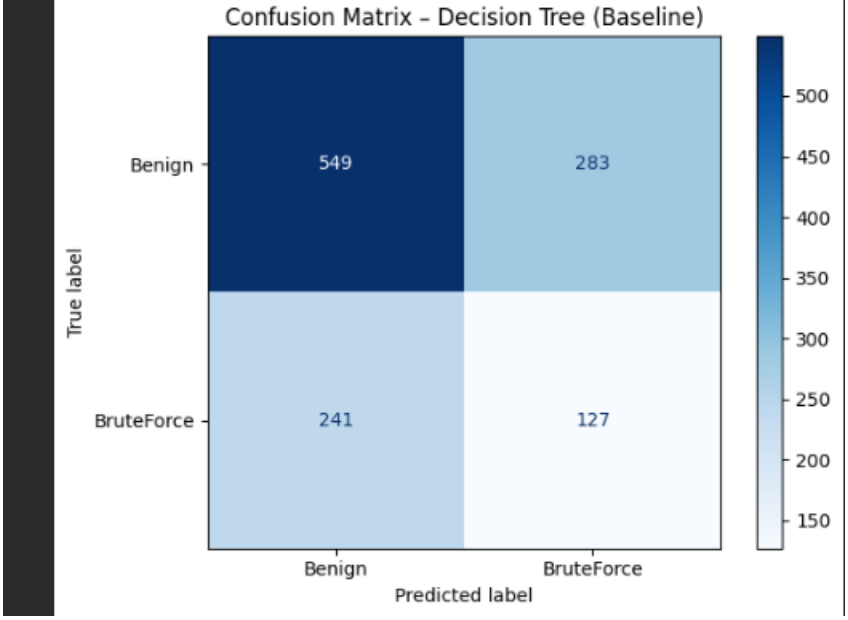
Consequences of the class imbalance identified earlier in the analysis.

However, on Benign traffic the model performed better, achieving a precision of 69%, recall 66% and f1-score of 68%.

```
=== Decision Tree (Baseline) ===
              precision    recall  f1-score   support

      Benign       0.69      0.66      0.68       832
  BruteForce       0.31      0.35      0.33       368

    accuracy                           0.56      1200
   macro avg       0.50      0.50      0.50      1200
weighted avg       0.58      0.56      0.57      1200
```



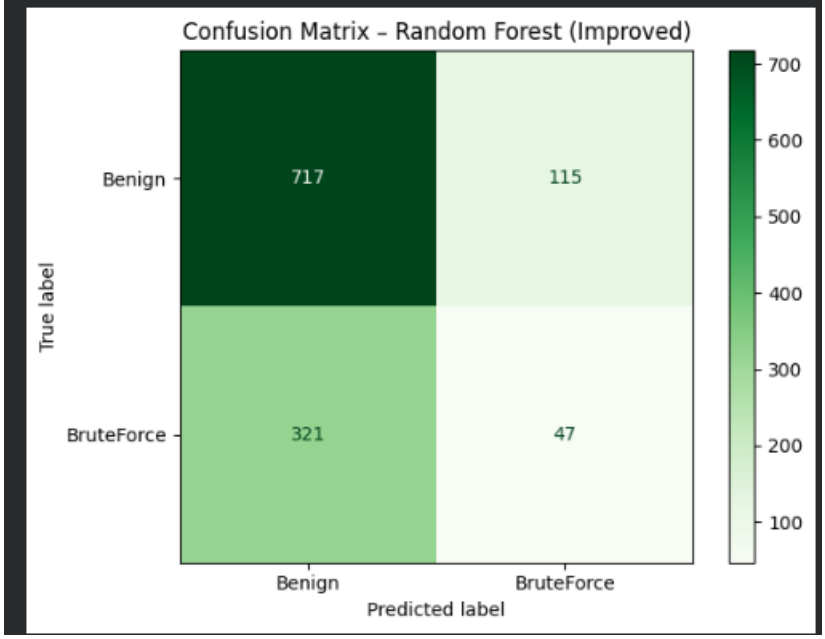
Model 2 Random Forest

Second model achieved an overall accuracy of 64%, Benign f1-score at 77% and Brute Force at 18% only. Seeing an increase in overall accuracy suggests that the model performs better, but Brute Force detection went down significantly.

```
=== Random Forest (Improved) ===
              precision    recall  f1-score   support

      Benign       0.69      0.86      0.77       832
  BruteForce       0.29      0.13      0.18       368

    accuracy                           0.64      1200
   macro avg       0.49      0.49      0.47      1200
weighted avg       0.57      0.64      0.59      1200
```



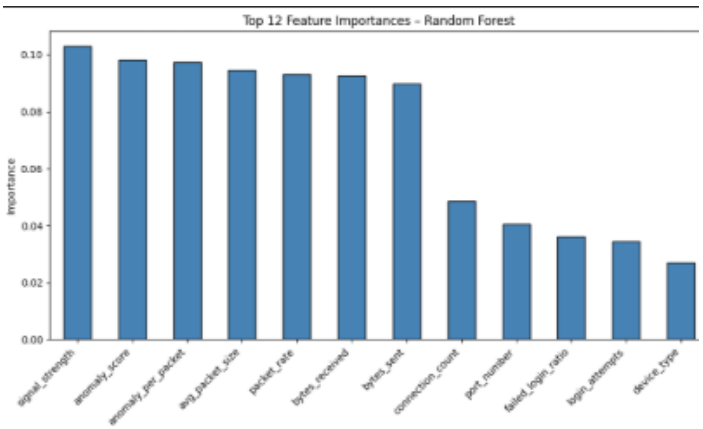
Brute Force
Precision – went down from 31% to 28%
Recall – went down to 13% from 35%
F1-score: went down from 33% to 18%

Model improved at identification of legitimate traffic rather than at Brute Force.

Out of 368 only 47 Brute Force attacks were predicted, returning 321 false negatives.

Majority class dominated here, demonstrating improvements for Benign traffic with an increase of f1-score from 68% to 77% and recall increase from 66% to 86%.

This highlights the need for class weight adjustments.



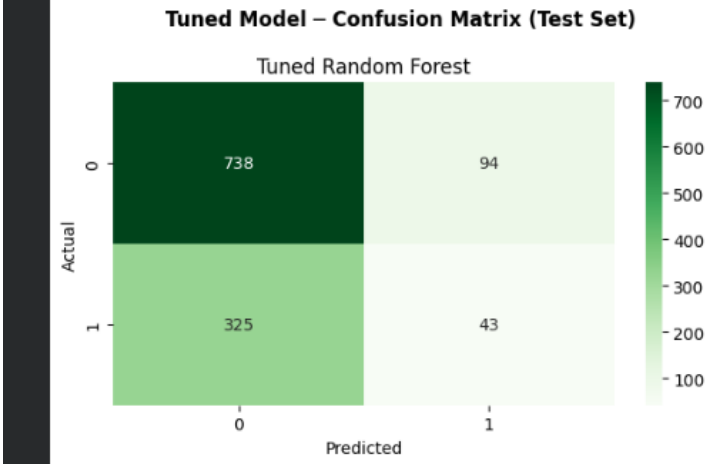
signal_strength, anomaly_score, anomaly_per_packet, avg_packet_score, packet_rate, bytes_received and bytes_sent influenced the Random Forest model. The engineered feature earlier anomaly_per_packet is valid for this dataset proving its existence.

Model 3 Tuned Random Forest

```
=== Tuned Random Forest – Test Set ===
  Estimators: 100
  Max Depth : None
  Min Samples Leaf: 10
              precision    recall  f1-score   support

      Benign       0.69      0.89      0.78       832
  BruteForce       0.31      0.12      0.17       368

    accuracy                           0.65      1200
   macro avg       0.50      0.50      0.47      1200
weighted avg       0.58      0.65      0.59      1200
```



Third model achieved an overall accuracy of 65%, Benign f1-score at 78% and Brute Force at 17% only. Seeing again a small increase in overall accuracy does not favor the model as statistics for Brute Force have fallen again.

Brute Force
Precision – went back up to 31%
Recall – went down to 12% from 35%
F1-score: went down from 33% to 17%

Tuned Random Forest shows improvement but still the problem remains valid, Brute Force attacks are missed It makes the model highly unreliable, with the root cause:

CLASS IMBALANCE

Optimal parameters were set as:

- Estimators = 100
- Depth = unlimited
- Min Samples Leaf = 10

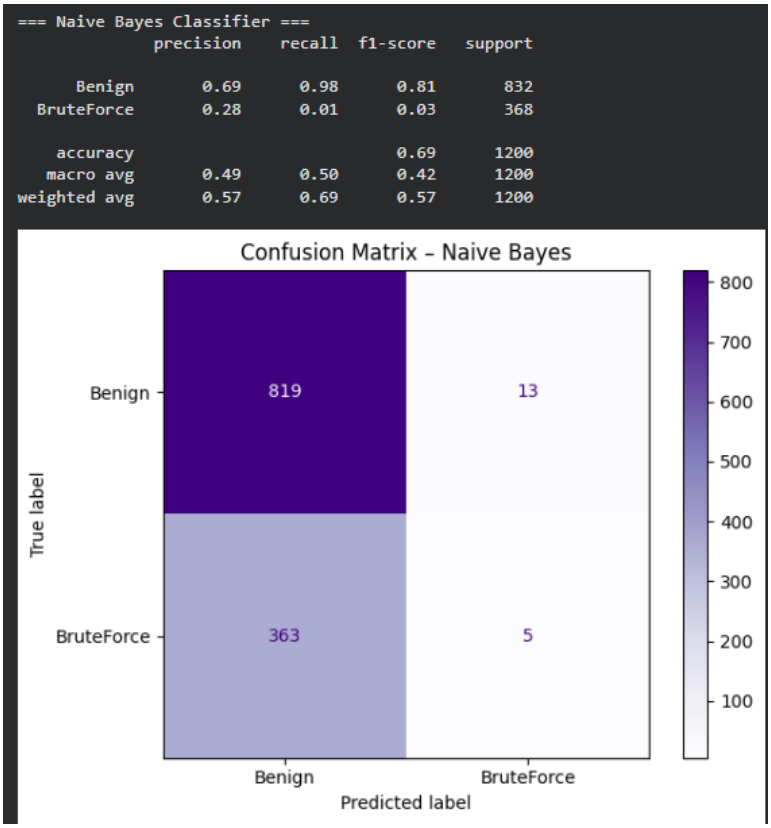
Correctly identified Brute Force dropped to 43, suggesting that tuning the model is not enough for overcoming the class imbalance problem. Other approaches should be implemented to increase the performance for reliable Brute Force detection eg. Resampling.

Model 4 Naive Bayes

Naïve Bayes achieved an overall accuracy of 69%, Benign f1-score at 81% and Brute Force at 3% only. Shocking how the model keeps improving its overall accuracy but our main problem of identifying the Brute Force activity keeps degenerating.

Brute Force
Precision – dropped to 28%
Recall – went down to 1% from 35%
F1-score: went down from 33% to 3%

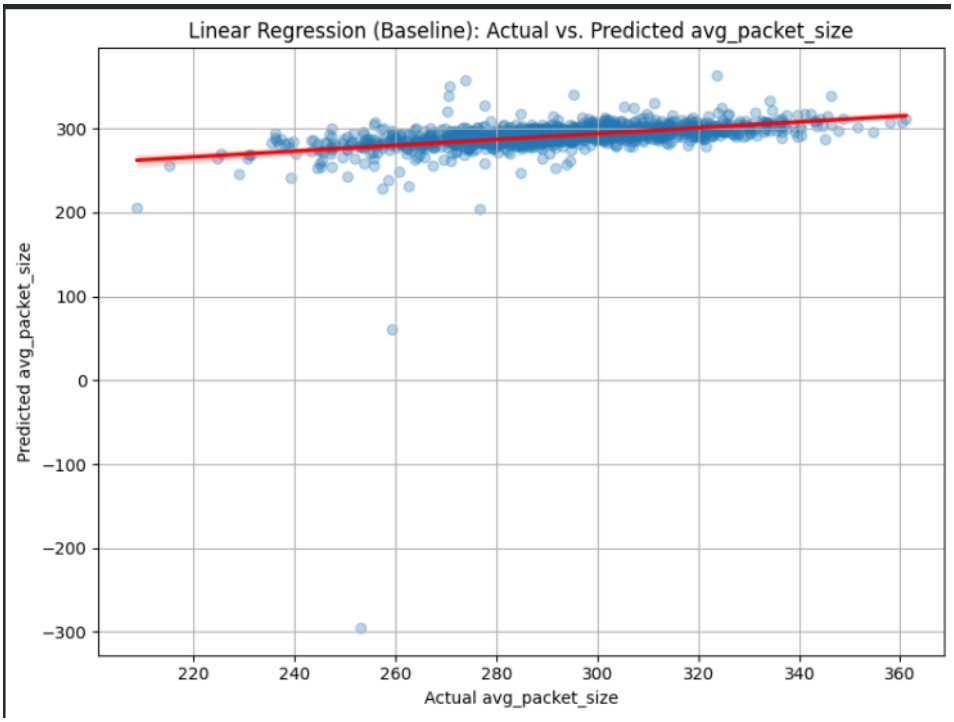
Only 5 Brute Force attacks were labeled as Brute Force out of 368, meaning that this model is useless in our case.



Regression

Baseline Regression

```
=== Linear Regression (Baseline) ===
MAE:  15.5046
RMSE: 25.4684
R2:   -0.2002
CV R2: mean=0.1922, std=0.1655
```



A poor performance of the baseline's linear regression, showing a weak prediction.

Model's predictions on average differ by 15.5 units from the actual values, but some errors are bigger (outliers) and bring the RMSE to ~25.5.

Predicting the mean would be more accurate than the model's predictions indicates the R2: - 0.2002

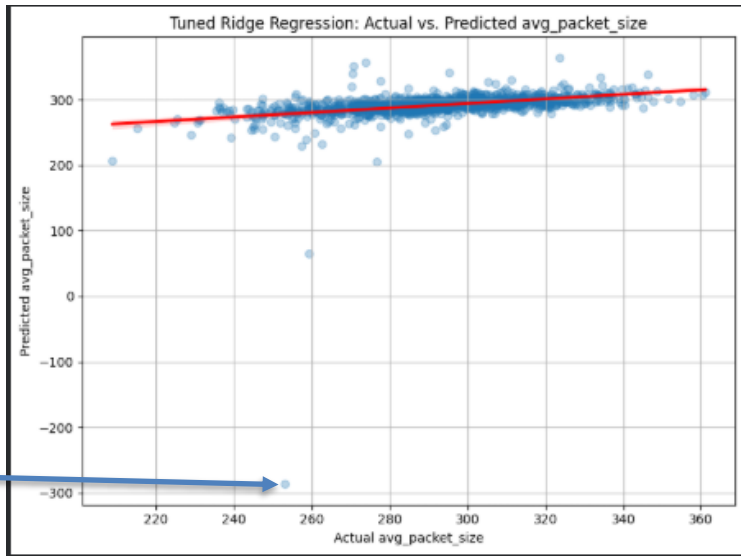
The R2 mean presents that the model is performing bad at predicting avg_packet_size making it unstable and not useful, also because the standard deviation (16%) is nearly as high as the mean.

Improved Regression

```
=== Ridge Regression (Improved) ===
MAE:  15.5043
RMSE: 25.4506
R2:   -0.1986
CV R2: mean=0.1924, std=0.1650
```

Ridge Regression shows only small improvement of R2 = -0.1986, but in the end both perform the same.

Showing same bad predictions where the actual value was ~ 255 it predicted at nearly -300.



Grid Search Regression

```
=== Ridge Regression – GridSearchCV Results ===
Best parameters: {'alpha': 10.0}
Best CV R2:      0.3064

Top 5 configurations:
 param_alpha  mean_test_score  std_test_score  mean_train_score
        10.0           0.3064          0.0505            0.3162
         1.0           0.3062          0.0516            0.3162
         0.1           0.3062          0.0517            0.3162
       100.0           0.3039          0.0433            0.3122
      1000.0           0.2469          0.0281            0.2527
```

Alpha = 10.0 was identified as the optimal parameter, with improvements of CV R2 mean of 0.3064 and standard deviation of 0.05. Looking better than the previous linear regression of ~0.19 results for the baseline and improved model. Also, Alpha = [1.0, 0.1, 100.0] are nearly as good as the optimal parameter with CV R2 mean results of 0.3062 and 0.3039.

Conclusion

This project required the application of fundamental principles and knowledge of AI and Data Science for analyses of a real-word cybersecurity-related dataset, following the CRISP-DM framework. The developed model can classify the network activity as Benign or Brute Force.

For model comparison, 4 approaches were implemented and evaluated:

- Baseline Decision Tree
- Random Forest
- Tuned Random Forest
- Naïve Bayes

Model	Accuracy	Precision (w)	Recall (w)	F1 (weighted)	CV F1 Mean	CV F1 Std
Decision Tree (Baseline)	0.5633	0.5768	0.5633	0.5695	0.5607	0.0105
Random Forest (Default)	0.6367	0.5679	0.6367	0.5861	0.5904	0.0127
Random Forest (Tuned)	0.6508	0.5776	0.6508	0.5923	0.5968	0.0095
Naive Bayes	0.6867	0.5656	0.6867	0.5718	0.5713	0.0042

Overall Comparison

Overall accuracy jumping from each model improved, starting from 56% - Baseline, 63% - Random Forest, Tuned Random Forest achieving an accuracy of 65% and Naïve Bayes – 68% indicating that each model is getting more accurate at predicting. The precision is similar, but recall is getting better at each model meaning that each model is predicting correctly. But in the end is failing and putting the system in danger because of missed Brute Force attacks.

With an F1-score of 59% the Tuned Random Forest indicates that optimizing the data using Grid Search gives improvements as it sets the best parameters to be used for better predictions, which can be seen by comparing the Default Random Forest and Tuned predictive capabilities.

Model	BF Precision	BF Recall	BF F1	False Negatives
Decision Tree (Baseline)	0.3098	0.3451	0.3265	241
Random Forest (Default)	0.2901	0.1277	0.1774	321
Random Forest (Tuned)	0.3139	0.1168	0.1703	325
Naive Bayes	0.2778	0.0136	0.0259	363

Brute Force Detection Comparison

However the main challenge was not solved, while the accuracy improved the Brute Force recall got worse. Predicting at the start 241 false negatives using the Baseline model and 363 using the Naïve Bayes model, an increase of 122 false negatives between the models (out of 368). In cybersecurity context: *Missing attacks > network in danger.*

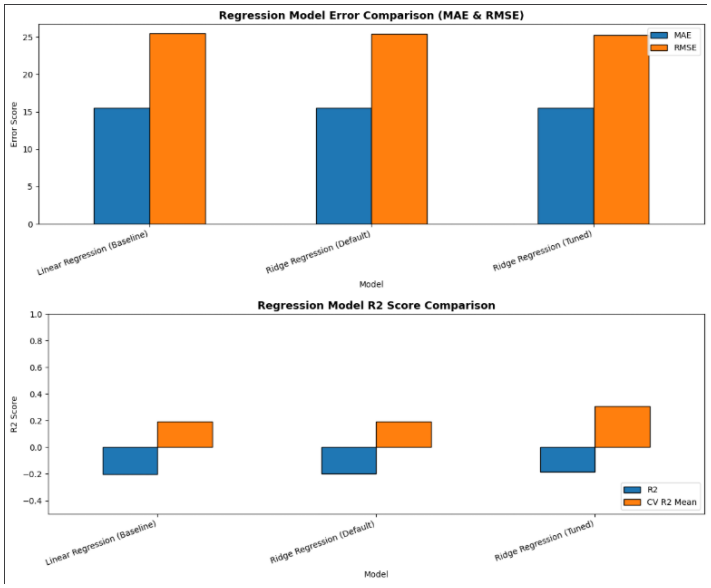
Overall, the Tuned Random Forest achieved better at accuracy but in relation to our presented challenge it dropped the performance. This brings back the main dataset problem identified earlier, the Class Imbalance, which favors the majority: Benign Traffic.

Not suitable for real world cybersecurity use, due to its low performance for detection of Brute Force sessions, therefore further improvements should be implemented starting by resampling (SMOTE) the target class and using more than one detection mechanism.

Regression Comparison

Model	MAE	RMSE	R2	CV R2 Mean	CV R2 Std
Linear Regression (Baseline)	15.5046	25.4684	-0.2002	0.1922	0.1655
Ridge Regression (Default)	15.5043	25.4506	-0.1986	0.1924	0.1650
Ridge Regression (Tuned)	15.5016	25.2941	-0.1839	0.3064	0.0505

The most stability is offered by the Grid Search Regression model (Tuned) as it searched for the most optimal parameter for the model to score high. Noticeable across all models the negative R2 and high RMSE indicating that it is not a linear relation between packet size and other features.



References

Verma, R. (2026). Lectures: ALL, COMPH2706, *Fundamentals of AI & Data Science, TUD.*

Verma, R. (2026). Class Demos: ALL, COMPH2706, *Fundamentals of AI & Data Science, TUD.*

(2026). Labs: ALL, COMPH2706, *Fundamentals of AI & Data Science, TUD.*

Collab Link - https://colab.research.google.com/drive/1j3CFUditbG9YIf7QiC6aEjT6-49hOG9s?usp=sharing